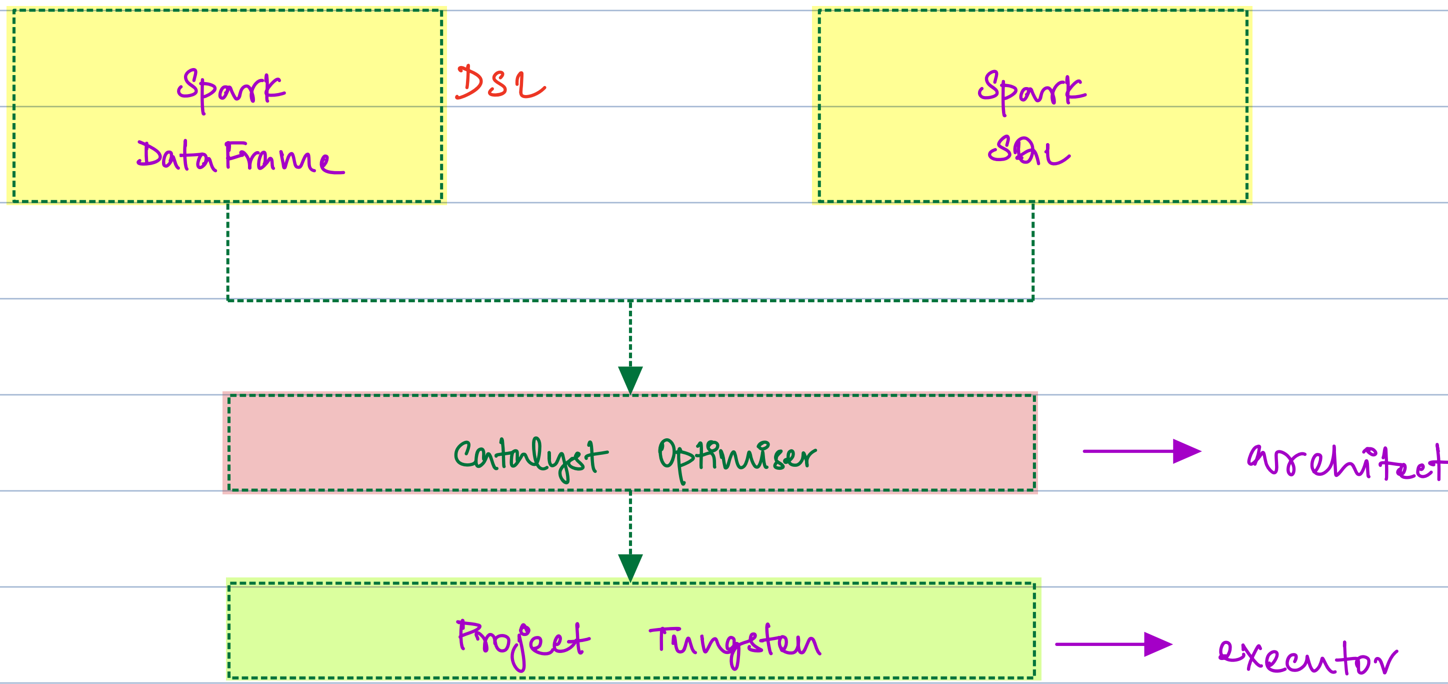
Agenda

1. Spark SQL
2. Demo - Action vs Transformation
3. Demo - Comedy Movie
4. Spark Catalyst Optimiser
5. Spark Tungsten Project

Spark SQL → High level End point



Spark SQL is a highlevel API that provides abstraction on top of your RDD. You can write your business logic using Domain specific language (DSL) or SQL which provides all essential function.

Catalyst Optimiser

Heart of spark SQL that provides performance enhancement. The catalyst optimiser processes query through sequence of stages to produce highly efficient physical plan.

Analysis → Logical optimisation → Physical planning → Code generation

Predicate Pushdown : where | Logical optimisation
Projection Pruning : SELECT columns |

Cost Based optimisation : cost model | Physical Planning

Project Tungsten Execution Engine

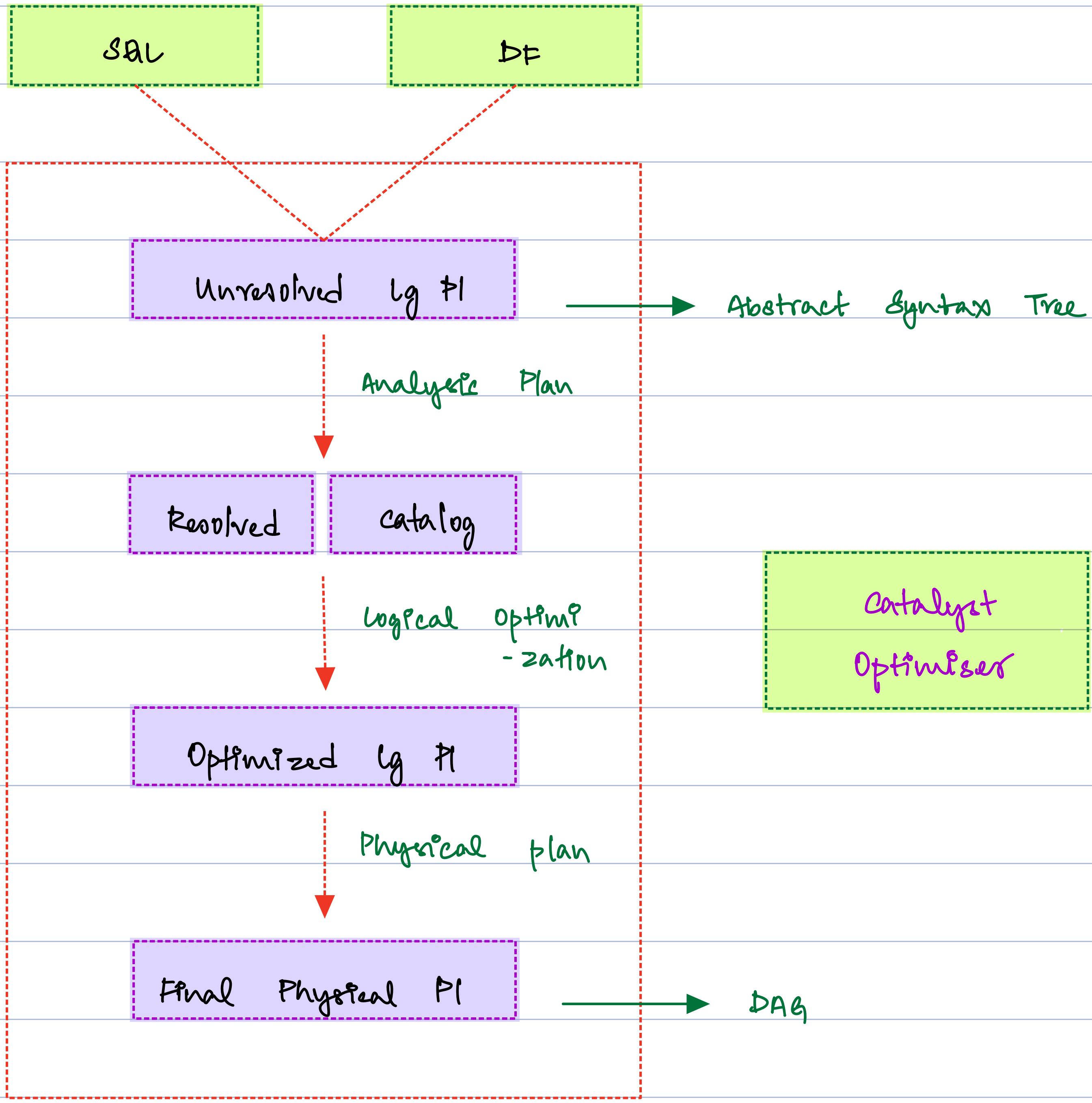
Tungsten is an intelligent Execution Engine. It has three pillars that optimises the Execution.

Off - Heap memory utilization. (avoids overload in JVM)

Better Caching Plan : uses different Datatype closes to CPU caching

Better serialization of data : Introduced better serialization & deserialization designs (unsafe Row)

Overall Flow



Final Physical Pl

1. Whole - Stage Code Gen
2. Optimised Bytecode Generation
3. UnsafeRow & Off - Heap Memory
4. Avoid GC OverHead

Tungsten Engine

How to create a DataFrames → spark immutable object with schema

① From external Data Sources → spark.read.*

② From in-memory collection → spark.createDataFrame

③ From RDD → rdd.toDF ([ ])

④ From SQL → spark.sql (" Query");

Demo

1. Perform Actions → .show() , .count() ,
   .write.format("parquet").save(path)

2. Narrow Transformation → .filter() , .select() , .drop()

3. Wide Transformation → groupBy() , .join() , .orderBy()